==========

CO2_ASSESSMENT TOOL 2

APPLICATION - BASED QUESTION_2Q

==========

Submitted in partial fulfilment for the course of

CSA0606-Design and Analysis of Algorithm

for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

By, Avinash M

(192521196)

UNDER THE GUIDANCE OF

Dr. Rajagopal K

SIMATS ENGINEERING

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

July 2026

Q51.Sorting Power Consumption Readings in a Smart Grid System

1. Analysis of the Given Scenario

A smart grid is a modern electricity distribution network that uses digital communication technology to monitor, control, and manage the flow of electricity between suppliers and consumers. In this scenario, thousands of smart meters installed across households, industries, and substations continuously transmit power consumption readings to a central data collection server. Because these readings arrive from geographically distributed devices over independent communication channels, they reach the central system in random, unordered sequence rather than in a fixed chronological or magnitude-based order.

For meaningful analysis such as identifying peak-load periods, detecting abnormal consumption, generating billing reports, and forecasting future demand, the raw readings must be sorted periodically — typically by timestamp, meter ID, or consumption value. Since the dataset is refreshed at regular intervals (for example every 15 minutes or every hour) and grows in size over time, the choice of sorting algorithm directly affects the responsiveness and scalability of the analytics engine. This assignment analyses two classical comparison-based sorting techniques, Selection Sort and Bubble Sort, in the context of this periodic sorting requirement, and extends the discussion into a complete system design covering requirements, technology stack, database design, implementation, and performance evaluation.

2. Algorithm Performance Analysis

a) Behaviour of Selection Sort and Bubble Sort on the Dataset

Selection Sort works by repeatedly selecting the minimum (or maximum) element from the unsorted portion of the array and placing it at the beginning of the sorted portion. For a smart-grid dataset of n meter readings, Selection Sort scans the entire unsorted sub-array in every pass to find the minimum value and performs exactly one swap per pass, regardless of how disordered or ordered the input already is. This means its comparison count is fixed at n(n-1)/2 for every possible input, making its behaviour highly predictable but insensitive to any existing order in the incoming readings.

Bubble Sort works by repeatedly comparing adjacent elements and swapping them if they are in the wrong order, causing larger values to “bubble” toward the end of the array with each pass. Since smart-meter readings arrive in random order, Bubble Sort in its naive form also performs close to n(n-1)/2 comparisons and up to n(n-1)/2 swaps in the worst case. However, a common optimisation — stopping early when a pass completes with no swaps — allows Bubble Sort to detect a nearly sorted or fully sorted batch of readings (which can happen if only a few new

readings are appended to an already sorted dataset) and finish in close to O(n) time, giving it an adaptive advantage that Selection Sort does not have.

b) Comparison of Efficiency: Comparisons and Swaps

The table below summarises the theoretical time complexity of both algorithms in terms of comparisons and swaps (data movements), which is the primary cost factor when sorting large volumes of streaming sensor data such as smart-grid readings.

Parameter	Selection Sort	Bubble Sort
Best Case Comparisons	$O(n^2)$ (always $n(n-1)/2$)	$O(n)$ (with flag optimisation)
Worst Case Comparisons	$O(n^2) = n(n-1)/2$	$O(n^2) = n(n-1)/2$
Best Case Swaps	$O(n)$ (at most n-1 swaps)	$O(1)$ (no swaps if sorted)
Worst Case Swaps	$O(n)$ (at most n-1 swaps)	$O(n^2)$ (up to $n(n-1)/2$)
Stability	Not Stable	Stable
Adaptive to Sorted Data	No	Yes
Space Complexity	$O(1)$	$O(1)$
Number of Data Movements	Low (few swaps)	High (frequent swaps)
Suitability for Smart-Grid Stream	Moderate	Poor for large batches

From the table, both algorithms share the same worst-case and average-case comparison complexity of $O(n^2)$, since both must examine essentially every pair of elements in an unordered array. The key practical difference lies in the number of swaps. Selection Sort performs at most n-1 swaps in total because it moves an element into its final position only once per pass, which makes it far more efficient when each swap operation is expensive — for example when a swap involves moving a large reading record containing meter ID, timestamp, voltage, and current values rather than a single number. Bubble Sort, in contrast, may perform a swap after almost every comparison, resulting in up to $n(n-1)/2$ swaps for reverse-ordered data, which increases memory write operations and can be costly for large, frequently updated smart-grid datasets.

On the other hand, Bubble Sort is a stable sort — it preserves the relative order of readings that have equal values (for example two meters reporting identical consumption at the same timestamp) — whereas Selection Sort is not guaranteed to be stable because of its long-distance swaps. Bubble Sort also benefits from the early-termination optimisation on partially sorted batches, which is

realistic in a smart grid where only the newest readings are unsorted while the historical portion of the dataset is already sorted from the previous cycle.

c) Recommendation and Justification

For the smart-grid scenario described, Selection Sort is recommended over Bubble Sort when the number of data swaps must be minimised, such as when each reading is a large composite record and memory-write cost dominates performance — Selection Sort guarantees at most n-1 swaps irrespective of input order, giving predictable and low write overhead.

However, if the periodic batches mostly contain a small number of new readings appended to an already-sorted historical dataset (which is the more realistic operating condition for a smart grid that sorts every 15 minutes), the optimised Bubble Sort with early termination is the more suitable choice, because it can detect the already-sorted portion and complete in close to linear time, reducing CPU cycles for the far more frequent case of near-sorted data.

In practice, for production-scale smart grid systems handling thousands to millions of readings, neither Selection Sort nor Bubble Sort is actually deployed, because both are $O(n^2)$ algorithms that do not scale. The realistic recommendation is to use an O(n log n) algorithm such as Merge Sort or an efficient library implementation such as Timsort (used internally by Python's sorted() and Java's Collections.sort() for objects), since Timsort is itself optimised to exploit already-sorted runs — combining the adaptiveness of Bubble Sort with the guaranteed O(n log n) worst-case bound. Selection Sort and Bubble Sort remain useful here mainly for small batches, teaching/demonstration purposes, embedded meters with extremely limited memory, or as a baseline for comparing against more advanced algorithms.

3. Identification of User Requirements

3.1 Functional Requirements

- Collect real-time power consumption readings from multiple smart meters.
- Store incoming readings with meter ID, timestamp, and consumption value.
- Periodically sort the readings (by timestamp, meter ID, or value) for analysis.
- Display sorted data and summary statistics on a web dashboard.
- Allow filtering of readings by date range, meter ID, or region.
- Generate consumption reports and peak-load alerts for administrators.
- Provide a login mechanism for administrators and grid operators.

3.2 Non-Functional Requirements

- Performance: sorting and dashboard refresh should complete within a few seconds for each cycle.
- Scalability: system should handle a growing number of smart meters and readings.
- Reliability: no reading should be lost or duplicated during collection or sorting.
- Security: only authenticated users should access consumption data and reports.
- Usability: the web interface should present sorted data in a clear, readable format.
- Maintainability: the codebase should be modular so the sorting module can be replaced or upgraded easily.

4. Suggested Technologies

Layer	Recommended Technology
Frontend (Webpage)	HTML5, CSS3, JavaScript, Bootstrap / React.js for a responsive dashboard
Backend / Server	Node.js (Express.js) or Python (Flask / Django) for REST APIs
Database	MySQL / PostgreSQL for structured relational storage of readings
Sorting / Analytics Module	Java or Python implementation of Selection Sort, Bubble Sort, and Merge Sort/Timsort
Data Visualisation	Chart.js or D3.js for graphs of consumption trends
Communication Protocol	MQTT / HTTP REST for transmitting meter readings to the server
Hosting / Deployment	Cloud platform such as AWS, Azure, or a local Apache/Nginx server

5. Design of the Webpage and Database

5.1 Webpage Design

The web dashboard is designed with a clean, single-page layout consisting of a navigation header, a filter panel, a sortable data table, and a summary chart area. Below is the conceptual page structure:

- Header: Smart Grid Monitoring Dashboard title with navigation links (Home, Readings, Reports, Login).
- Filter Panel: dropdowns/date-pickers to filter by meter ID, region, and time range.
- Data Table: displays meter ID, timestamp, and consumption value in sorted order, with a control to choose the sorting field and algorithm.
- Chart Section: line/bar chart showing consumption trends over the selected period.
- Footer: system status, last sort/update time, and contact/admin information.

5.2 Database Design

The database is designed with three core tables — Meter, Reading, and User — to normalise data and avoid redundancy.

Table: Meter	Description
meter_id (PK)	Unique identifier for each smart meter
location	Installation address / region of the meter
meter_type	Residential, commercial, or industrial

Table: Reading	Description
reading_id (PK)	Unique identifier for each recorded reading
meter_id (FK)	References Meter table
timestamp	Date and time the reading was captured
consumption_value	Power consumption in kWh
sorted_flag	Indicates whether the reading has been included in the last sort cycle

Table: User	Description
user_id (PK)	Unique identifier for each system user
username	Login name of the administrator/operator
password_hash	Encrypted password for authentication
role	Admin, Operator, or Viewer

6. Implementation Steps

1. Set up the database schema in MySQL/PostgreSQL with the Meter, Reading, and User tables described above.

2. Develop backend REST APIs (Node.js/Express or Python/Flask) for receiving meter readings, storing them in the database, and retrieving sorted results.

3. Implement the sorting module as a separate service/function that can perform Selection Sort, Bubble Sort, or an optimised algorithm (Merge Sort/Timsort) based on configuration or dataset size.

4. Schedule the sorting process periodically (e.g., using a cron job or a timer in the backend) to sort newly collected readings every fixed interval.

5. Design and build the frontend dashboard using HTML/CSS/JavaScript (or React) with a table component bound to the sorted API results.

6. Integrate Chart.js/D3.js to visualise sorted consumption data as trend graphs.

7. Implement authentication (login/session or JWT tokens) to restrict dashboard access to authorised users.

8. Test the system with sample datasets of varying sizes to validate sorting correctness and measure execution time.

9. Deploy the application on a cloud server or local server and monitor performance under real/simulated meter data loads.

7. Justification of Design Decisions

- A relational database (MySQL/PostgreSQL) is chosen because meter readings are highly structured and benefit from indexing on timestamp and meter_id, which speeds up both storage and the periodic sorting queries.
- Separating the sorting logic into its own module allows the algorithm (Selection Sort, Bubble Sort, or Merge Sort) to be swapped without changing the rest of the system — directly supporting the comparison required in this assignment.
- REST APIs decouple the frontend and backend, allowing the dashboard to be updated independently and enabling future integration with mobile apps or third-party analytics tools.
- Using MQTT/HTTP for meter communication reflects real-world IoT smart-grid architectures, where lightweight protocols are preferred for constrained devices.
- Role-based authentication (Admin/Operator/Viewer) ensures data security while still allowing different stakeholders appropriate levels of access.

8. Performance Evaluation

To evaluate the two sorting algorithms in the context of this system, sample datasets of varying sizes (n = 100, 500, 1000, 5000 readings) can be sorted using both Selection Sort and Bubble Sort, and their execution time, comparison count, and swap count recorded. The expected trend, based on their theoretical complexity, is summarised below:

Dataset Size (n)	Expected Relative Time (Selection Sort vs Bubble Sort)
100	Comparable; both complete quickly (milliseconds)
500	Selection Sort slightly faster due to fewer swaps
1000	Selection Sort noticeably faster; Bubble Sort swap overhead grows
5000	Both become impractical; a scalable O(n log n) algorithm is needed

This evaluation confirms that while Selection Sort consistently outperforms Bubble Sort on random smart-grid data due to fewer swaps, both algorithms degrade quadratically as the dataset grows, reinforcing the earlier recommendation that a production system should transition to Merge Sort or Timsort once the number of readings per sorting cycle becomes large (typically beyond a few thousand records).

9. Conclusion

This assignment analysed the problem of periodically sorting randomly arriving power consumption readings in a smart grid system. Selection Sort and Bubble Sort were compared in terms of comparisons, swaps, stability, and adaptiveness, showing that Selection Sort minimises data movement while Bubble Sort can exploit partially sorted data through early termination. For small or memory-sensitive batches, Selection Sort is preferable, while for near-sorted periodic batches, an optimised Bubble Sort performs better; for large-scale, production smart-grid deployments, a more advanced O(n log n) algorithm such as Merge Sort or Timsort is ultimately recommended. Beyond the algorithmic analysis, a complete system design was proposed, covering user requirements, technology stack, webpage and database design, implementation steps, and performance evaluation, providing a practical blueprint for building a scalable smart grid monitoring and analytics platform.